



Introduction to Cyber Security (IE2092)
2nd Year, 2nd Semester

Individual Assignment

Securing the Software Supply Chain: The Evolution of Vulnerabilities and the Adoption of Software Bill of Materials (SBOMs)

Submitted to

Sri Lanka Institute of Information Technology

IT24100139

Pinthu D.I.U.

In partial fulfillment of the requirements for the
Bachelor of Science Special Honors Degree in Information Technology

18 April 2026

Table of Contents

1. Abstract	1
2. Introduction.....	1
3. Topic Development.....	3
3.1. Trust models in software early and the narrow perspective of software security.....	4
3.2. Growth of Open-source utilization and dependency risk.	4
3.3. Supply chain attacks are considered strategic threats.	5
3.4. Vendor compromise to ecosystem compromise.	5
3.5. Development of ordered defensive systems.	7
3.6. Current Landscape and Transition to the Future.....	8
4. Future Developments in the Area	9
4.1. Greater adoption of software transparency standards and SBOMs.	9
4.2. More effective provenance, attestations, and verification of artifacts.	10
4.3. Security controls are built within CI/CD pipelines.....	10
4.4. Automated dependency trust score and risk-based adoption.....	11
4.5. More focus on safe development maturity and supplier assurance.	11
4.6. Future prospects in the field in general.....	12
5. Conclusion	13
References.....	14

1. Abstract

The security of software supply chains has become a significant branch of cybersecurity since, nowadays, software is not developed using any in-house code, but relies on open-source libraries, third-party components, package repositories, cloud services, and automated DevSecOps CI/CD pipelines. This increased dependency has provided new attack surfaces, with a single vulnerability in a dependency, a build system, or an update system being able to impact thousands of end-users. The recent events have revealed that criminals start to use trusted processes of software delivery more frequently instead of directly attacking end-users, and software supply chain security is becoming a burning issue for governments, organizations, developers, and end-users. This report examines the shift in software supply chain security where the previous protection of the codes-based is replaced with an overall model focusing on dependency management, integrity of a build, artifact signing, provenance and transparency. It mentions the increasing significance of secure software development methods, Software Bills of Materials (SBOMs), signed attestations, and the recently developed frameworks like the Secure Software Development Framework (SSDF), SLSA, Sigstore, and the OpenSSF Scorecard [1] [2] [3] [4]. The report also examines existing problems in securing open-source ecosystems and the CI/CD pipelines and outlines future trends that are likely to influence this area. In totality, the paper asserts that the supply chain security of software is becoming a necessary requirement of a trustworthy software delivery, and that increased visibility, verification and responsibility across the software lifecycle will be the key to resiliency in the future.

2. Introduction

Software supply chain security has become one of the most important new areas in cybersecurity, as modern software is seldom created, assembled and distributed within a unique confined area of trust. Instead, modern software applications rely on complex supply chains that encompasses both in-house code, open-source libraries, third-party dependencies, package repositories, build server, testing frameworks, container images, cloud services, and continuous integration and continuous delivery pipes. The guidelines provided by NIST on the topic of software supply chain security in DevSecOps pipelines explicitly address each of the stages of the pipeline [5] as a part of the software supply chain, demonstrating that the attack surface goes much beyond the end application itself. Simultaneously, CISA defines a Software Bill of Materials as a list of software ingredients [6], and emphasizes that nowadays software is often a layered product and is made of numerous interconnected elements instead of a single self-contained object.

This growing interdependence has brought about newer avenues for the attackers. Opposed to attacking end users or directly exploiting vulnerabilities in deployed applications, adversaries today target processes, dependencies, and trust relationships that enable software. A software supply chain attack involves the attacker trying to gain control over a trusted component of

the development or delivery chain to allow malicious code or component modifications or unauthorized access to propagate to downstream users via otherwise legitimate channels. In its alert about the SolarWinds compromise, CISA explained that a threat actor breached the Orion software supply chain [7], and their subsequent alerts about both the 3CXDesktopApp attack [8] and the lessons learned in the XZ Utils incident also serve to illustrate how attackers can exploit [9] the trust of vendors, update systems, and even open-source maintainer ecosystems. Such incidents have elevated security in the software supply chain to a significant issue not only to software vendors, but also to governments, enterprises, developers, and just ordinary users who trust the usage of trusted software products daily.

Modern Software Development Supply Chain Overview

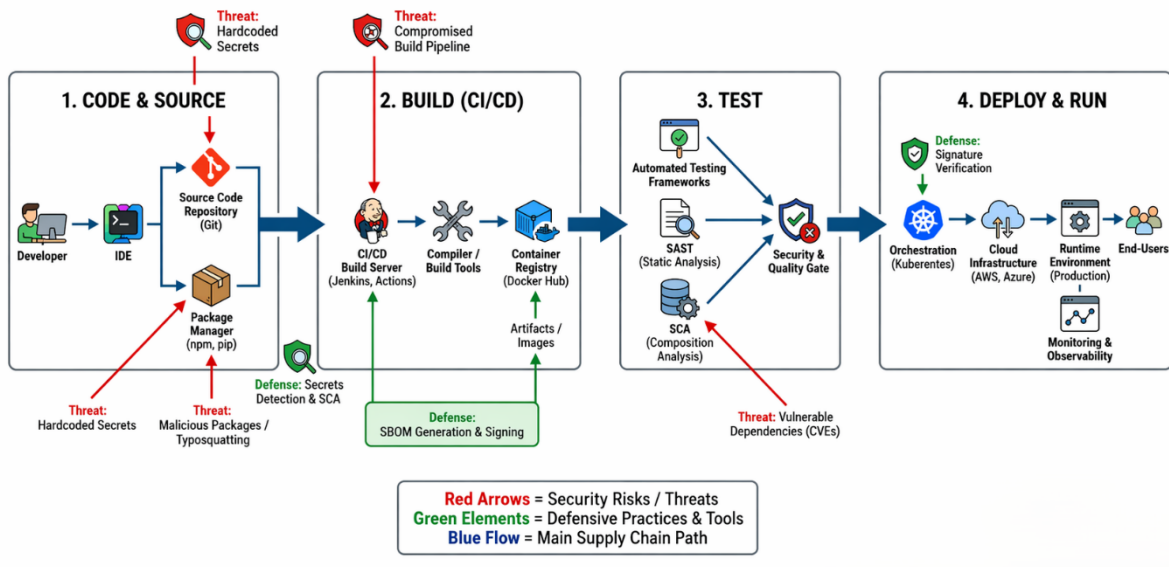


Figure 1: Supply chain overview diagram

This topic has become even more significant due to the fact that organizations are currently dependent on the use of open-source software and automated delivery pipelines in order to deliver speed, scalability, and innovation. Although automated DevSecOps pipelines provide great speed and operational efficiency, they also conceal the provenance, integrity and provenance of the assembled artifacts. The development of the supply chain has now included Infrastructure as Code (IaC). The tools used by development teams in the provision of cloud resources declaratively include Terraform and Ansible across different platforms including AWS and Azure. In turn, the CI/CD pipeline (i.e. GitHub Actions or Jenkins runners) obtains a very privileged, credentialed access to the underlying infrastructure. When an attacker gets access to the build server or finds hardcoded secrets in an IaC repository, they can self-provision rogue architecture, steal data, or modify deployment settings and shift the breakage point of the app being compiled to the pipeline itself. Due to this fact, the concept of software supply chain security is no longer restricted to secure coding itself. Dependency governance, protecting the integrity of the source, protecting the build environment, artifact signing, provenance, vulnerability management, and disclosing software composition have since been added. This broader perspective can be seen in the approach of NIST to secure software development, namely its Secure Software Development Framework, which suggests a core of high-level secure software development practices [1] that are

applicable to software development life cycle models to reduce the potential risk of software vulnerabilities. The trend is also observable in the new security architectures like SLSA, which works towards eliminating tampering [2], enhancing integrity, and securing packages and infrastructure through the entire software supply chain.

The other reason why this subject is particularly topical nowadays is that software supply chain security is an area that incorporates the aspects of technical, organizational, and policy. Security is not something that can be attained by relying on a popular package, a maintainer known to be reliable, or a reputable vendor. The evidence must be more and more the basis of trust. This is the reason why new methodologies put emphasis on Software Bills of Materials, provenance, attestations, and structured assurance models. The SBOM guidance provided by CISA reveals that component visibility is no longer regarded as a cornerstone [6] of managing software risk, and NIST SP 800-204D proves that software supply chain security measures should be part and parcel of CI/CD pipelines [5] and should not be considered an afterthought anymore. That is, the field is shifting towards informal trust to verifiable and measurable trust.

This report offers an overview of software supply chain security, an emerging cybersecurity area. It sets its boundaries, exposes the growing importance, documents the evolution from traditional code-centric approaches to broader views involving third-party components, repository security, build and CI/CD pipelines, software provenance, software artifact integrity, and it examines ongoing challenges including vulnerability of the open-source ecosystem, dependency vulnerabilities, lack of visibility of components, and inadequate build and release verification. It also explores future opportunities: expanded use of SBOM, better provenance, more secure CI/CD, and automated trust assessment; and discusses the implications of these for software vendors and users. The focus is on the security of the software development and delivery ecosystem (not deployed apps), open-source and third-party component risks, dependency security, build integrity, composition transparency, and verification practices (such as SSDF, SBOM and SLSA). This analysis suggests the report that supply chain security is not just a passing phenomenon, but essential for safe and robust systems, and that increased ecosystem complexity will require companies to ensure transparency, accountability and verification of guarantees through key lifecycle phases.

3. Topic Development.

Software supply chain security was not a one-dimensional cybersecurity problem. It evolved in parallel with the transformation of software engineering methodologies. In the past phase of software development, numerous organizations primarily concentrated on safeguarding their respective source code, internal networks, and published applications. Secure coding, patching vulnerabilities, access control, and perimeter defense were some of the main aspects of security work. Nonetheless, it became an inadequate model because software production evolved to rely more on external libraries, open-source elements, package managers, cloud services, automated build systems, continuous integration pipelines, and continuous delivery pipelines. The build, test, package, and deploy phases of CI/CD are now explicitly considered by NIST as the software supply chain [5], demonstrating that the scope of software security

should extend beyond the application being created to all of the processes and components of the software supply chain. Meanwhile, CISA defines a Software Bill of Materials as a nested inventory of software elements [6], which is a truth in the fact that modern software is built out of a large number of interdependent components, rather than created through the lens of a single independent object.

3.1. Trust models in software early and the narrow perspective of software security.

The initial phase in the history of software supply chain security featured a comparatively small trust model. Software would often be considered reliable when a known vendor, an internal development team, or an external repository provided it. The concept of security dialogue was more of the vulnerabilities found in the applications and less about the integrity of the ecosystem that produced them. In practice this meant that developers and organizations would usually assume that external packages and build tools, and update channels were innocent until there was explicit evidence to the contrary. This supposition was weakened by the passage of time as the processes involved in development became more distributed and software production increasingly depended on the use of third-party elements. This change is reflected in the SSDF by NIST, which states that secure software development practices must be part of software development life cycles [1] since many of the development models do not provide a specific focus on software security at a sufficiently detailed level. That is, the field has developed into a more formal and evidence-based security model out of an implicit trust model.

3.2. Growth of Open-source utilization and dependency risk.

A critical moment for software was the rapid rise in the use of open-source software and reuse of dependencies, which dramatically sped up the work of software engineering but also created catastrophic risks of transitive dependencies, such as via public package managers such as 'npm', PyPI' and 'Docker Hub'. Today's microservice development involves intricate dependency chains, so an infected or typo squatted library can easily infect thousands of automated builds, leveraging dependency resolution to infect systems. This approach improved development efficiency but also increased the threat surface beyond the corporate network's perimeter to the defense in depth of the infrastructure itself. For example, leveraging an unauthenticated base image to create a container embeds binaries at the runtime level; if orchestrated in Kubernetes, it's easier to move laterally across the network, bypassing firewalls. This can endanger many downstream applications. This is the same "visibility" issue CISA's SBOM and NIST's DevSecOps approach to software supply chain security seeks to resolve: security no longer just protects code, it now protects the origin, integrity, composition and risk of code. Trust could no longer be based on vendor reputation or popularity of an open source project, it also needed to be verifiable - in terms of origin, build, changes and composition of parts. This need for verifiable information and assurance gave birth to software supply chain security as a sub-discipline, and standards such as SBOM,

SSDF and SLSA because the former "trust until proven" approach was too risky to operate with these days.

3.3. Supply chain attacks are considered strategic threats.

The second significant step in the development of the discussion was the emergence of supply chain attacks as a high-impact strategic threat, as opposed to a technical niche issue. The 2020 incidents analyzed by ENISA, threatening to supply chain attacks, found that attacks on supply chains had grown [10] in both size and complexity, and estimated that by 2021, there would be four times as many attacks on supply chains compared to 2020. This was a significant shift in the awareness of cybersecurity since it demonstrated that attackers were becoming increasingly inclined to target suppliers, dependencies, and trusted channels as methods of reaching numerous targets simultaneously. Attackers might compromise one trusted upstream element and have the software ecosystem spread the compromise on their behalf, instead of having to attack each victim individually.

The SolarWinds Orion breach turned out to be one of the most significant steps in this development. According to CISA, a more sophisticated perpetrator of cybercrime had compromised the SolarWinds Orion software supply chain [7]. As this event showed, even a reputable system of vendor updates can be transformed into a massive distribution channel of malware. The importance of SolarWinds was not the technical level of the attack, but the fact that the attack revealed the vulnerability of traditional assumptions of trust. Those organizations that had acquired software via legal means still suffered as the compromise was made on the upstream side. Following this event, software supply chain security has become a topic of cybersecurity policy, purchasing, and secure development that were brought into focus a lot more.

3.4. Vendor compromise to ecosystem compromise.

Following SolarWinds, the information kept on developing with more attacks that broadened the community's insight into the ways supply chain attacks could take place. In 2023, CISA raised an alert on a supply chain attack on 3CXDesktopApp, saying that the legitimate voice and video conferencing software had been trojanized [8] and may result in multistage attacks. This case strengthened a central lesson that even apparently legitimate software signed might be used to deliver malicious code in the event that the upstream development process, or the release process, is corrupted. The attack thus increased the understanding that greater verification of the build artifacts, release integrity, and software provenance is necessary in the software supply chain security.

A new important dimension of development of the topic in 2024 became the XZ Utils incident. Speaking about the lessons of XZ Utils, CISA emphasized the need to code review, security scan, isolate builds, and have a more sustainable open-source ecosystem. [9] One particularly significant aspect of XZ Utils was that it emphasized the contribution of social and organizational weakness to the supply chain security. It was not just a matter of a technical issue, but also it demonstrated the vulnerability of maintainer trust, a lack of

control, and the weakness of ecosystems of critical open-source initiatives. This increased the area of the domain to extend further than merely a technical point of view and proved that the security of the software supply chain also depends on the project governance and community resilience as well as maintainers being sustained.

Combined, SolarWinds, 3CX, and XZ Utils demonstrate the expanding parameter of software supply chain security. The area shifted away from the issue and worry of bad updates to vendors, to worry about any weak link in the software production chain, including dependencies, maintainers, build systems, artifact repositories, and release mechanisms. It was a ground-breaking departure since it rendered it clear that secure software cannot be acquired by merely securing application code when it is developed. It should rather be backed through integrity controls throughout the software lifecycle.

Year	Incident	Attack Point	Main impact	Security Lesson	Source
2020	SolarWinds Orion compromise.	Vendors build and software update chain.	Malicious code was distributed through trusted Orion updates to many downstream organizations.	Trust in vendor updates alone is not enough; organizations need stronger build integrity, provenance, monitoring, and verification of released software.	(CISA) [7]
2023	3CXDesktopApp attack.	Legitimate desktop application release process.	A trojanized version of a trusted communication application was used to deliver further malicious activity.	Signed or well-known software can still be dangerous if the upstream release path is compromised; artifact verification and release-chain security are essential.	(CISA) [8]
2024	XZ Utils compromise.	Open-source maintainer trust and development pipeline.	A backdoor attempt in a widely used open-source component highlighted ecosystem fragility and the risk of maintainer-targeted compromise.	Software supply chain security also depends on project governance, review quality, sustainable maintainer support, and build isolation.	(CISA) [9]

3.5. Development of ordered defensive systems.

With the emergence of the threat, the field was transforming more towards responding to incidents through reactive incidents to structured defensive frameworks and assurances. The work of NIST on the Secure Software Development Framework was one of the most influential [1]. SSDF offers a fundamental collection of secure software development disciplines at a high level, which can be incorporated in the development lifecycle to diminish the chances of software vulnerabilities. The significance of it in the development of the subject is that it shifted the discourse further out of the realm of ad hoc best practices and more towards a structured framework that software vendors and buyers could reference. It also served to tie the software supply chain security together with overall secure development and software assurance objectives.

The emergence of SBOM as a convenient tool of software transparency was another significant event. CISA defines SBOM as a list of components that constitute software [6], and subsequent advice further defines SBOM as a basis of software component transparency. SBOM did not resolve all of the supply chain issues, but it made a tremendous impact on the field by making component visibility a key value. It was a significant conceptual move: the questions were not only whether software works, but whether it can be seen, reviewed, and maintained over the course of its existence.

This would later be expanded by SLSA in terms of tamper resistance, integrity, and package and infrastructure protection. The official documentation of the SLSA states that it is a security structure and a checklist of controls [2] and standards to stop tampering, enhance integrity, and protect packages and infrastructure. What is significant about its level-based approach is the fact that it acknowledges the reality that software supply chain security maturity can be incrementally improved, but not in one leap. The change here is considerable: software assurance started to shift to quantifiable maturity tracks, provenance, and build requirements that can be verified.

Simultaneously, Sigstore and in-toto served to reinforce the factual aspect of verification. The documentation above explains that artifacts can be verified more easily and safely using keyless signing and transparency logs and that Rekor is an auditable list of signing events. [3] [11] In-toto identifies itself as the framework to ensure the integrity of the supply chains of software [12]. These tools are a sign of the next phase of development of the topic: leaving the policy language of trust to technical, which enables organizations to check what was constructed, by whom, and in what conditions.

OpenSSF Scorecard is a further step towards such evolution, ensuring that users evaluate the risks of open-source projects [4] in security matters with the help of automated verification. This is important as the discipline no longer has an issue with reacting to post-compromise. It is also growing very much worried about the assessment of health and trust stance of dependencies prior to adoption. It is an obvious indication of the maturity of the subject: software supply chain security has ceased to be a reaction to incidents, and has become a peri-adoption risk-assessment and ongoing trust-testing.

Framework/ Tool	Main Purpose	Contribution to software	Source
SSDF	Provide secure software development practices	Gives organizations a structured set of practices to reduce software vulnerabilities across the development lifecycle	(NIST Computer Security Resource Center) [1]
SBOM	Provide visibility into software components	Improves transparency by showing what components, libraries, and dependencies are inside a software product	(CISA) [6]
SLSA	Improve integrity and resist tampering across the supply chain	Introduces a maturity-based approach for stronger provenance, build assurance, and supply chain security controls	(SLSA) [9]
Sigstore	Sign and verify software artifacts more securely and conveniently	Strengthens artifact trust through modern signing methods and transparency-log-based verification	(Sigstore) [3]
In-toto	Protect the integrity of steps in the software supply chain	Verifies that each stage in the chain is performed as expected and helps detect unauthorized changes or tampering	(in-toto) [12]
OpenSSF Scorecard	Assess open-source projects using automated security checks	Helps maintainers and users evaluate the security posture and trustworthiness of dependencies before adoption	(openssf.org) [4]

3.6. Current Landscape and Transition to the Future.

Today, software supply chain security is acknowledged as a critical component of contemporary cybersecurity, covering the software supply chain from integrating to deploying software artifacts. Motivated by the threats posed by rapidly evolving supply chain attacks and the ineffectiveness of implicit trust, the industry has successfully put in place basic defensive structures. As outlined in recent NIST guidance [5], attention has now clearly shifted to ensure critical CI/CD pipelines by relying on transparency and verification tools such as SBOM, SLSA, Sigstore and OpenSSF Scorecard.

But though these systematic frameworks and technical verification processes are now widely accepted, the day-to-day reality has evolved from creating such standards to enforcing them in practice. With increasingly complex and interwoven software supply chains, the need to move from manual evaluation to automated continuous policy enforcement is the stepping-stone for the next wave of developments in this space.

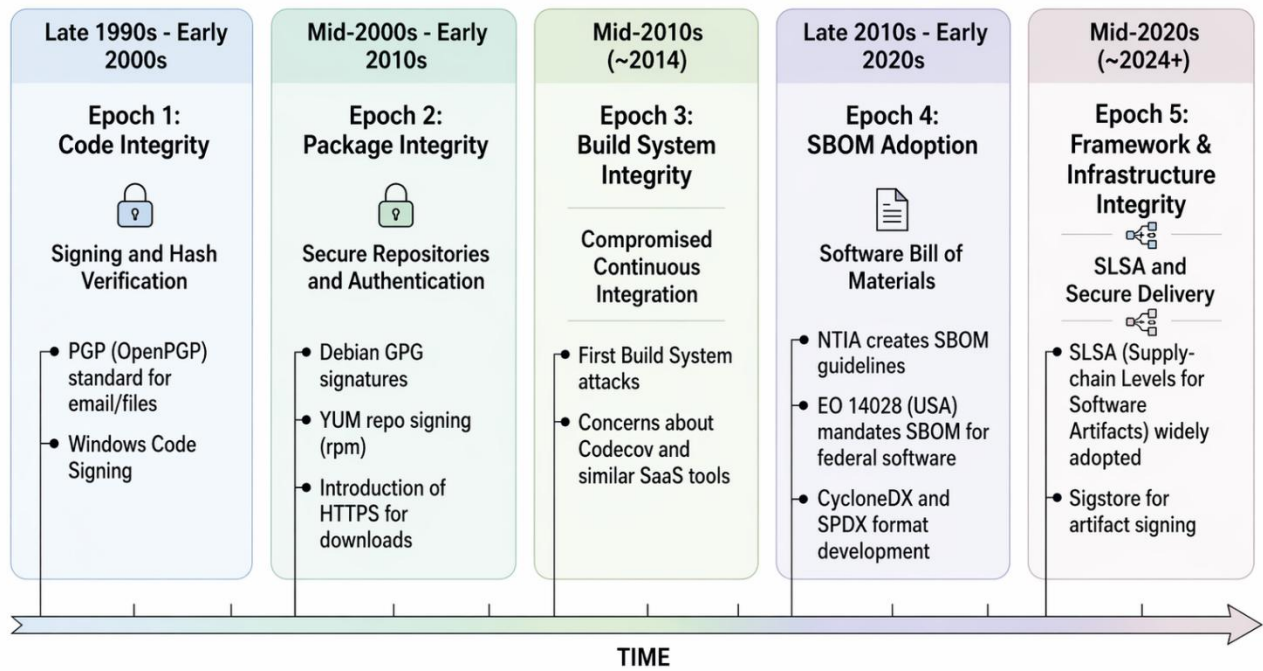


Figure 2: Evolution timeline of software supply chain security

4. Future Developments in the Area

The state of software supply chain security is highly dynamic, and its future will be influenced by the evolution of today's complex software ecosystems, the growing people's reliance on open-source projects, and the need to develop verifiable confidence in the software build and release process. According to recent advice by NIST, CISA, SLSA, Sigstore, and OpenSSF, the trend is shifting toward [1] [6] [2] [3] [4] the field based both on less formal trust and more on robust transparency, automation, and policy-based verification. Software supply chain security will change in the coming years with broader use of SBOM, improved provenance and attestation practices, enhanced incorporation of security controls into CI/CD pipelines, and more formal ways of evaluating the trustworthiness of open-source dependencies. The importance of these developments lies in the fact that future software security shall not just be based on vulnerability avoidance, but also provide a demonstration of the integrity, origin, and security posture of software throughout its lifecycle.

4.1. Greater adoption of software transparency standards and SBOMs.

The broader adoption of Software Bills of Material and more mature standards of software transparency are among the most significant future developments in this domain. According to CISA, an SBOM is one such tool to provide greater insight into software parts; its 2025 update to the minimum SBOM components reflects [6] that the concept of a simple list of ingredients is being replaced with more viable guidance on risk management of software. This means that the SBOMs will tend to be more standard, more detailed, and more effective in operational security decisions. While the generation of SBOMs is being normalized, the future of securing supply chains will be the automation of SBOM ingest and ongoing analysis. Standards such as CycloneDX and SPDX, in machine-readable formats, will play a

significant role in DevOps: seamlessly integrated with the Vulnerability Exploitability eXchange (VEX) format. VEX provides real-world context by showing not only whether a vulnerable library is used, but if it's vulnerable in the specific runtime environment. This dynamic, real-time analysis dramatically reduces false positives and alert fatigue. This is crucial because many supply chain risks arise from non-transparency of dependencies and transitive components: without knowing what we're using, an organization cannot swiftly assess its risks when a component becomes compromised. Transparency must therefore be the norm - the default - for software provisioning and delivery.

4.2. More effective provenance, attestations, and verification of artifacts.

The second significant future change will be the increased application of provenance, attestations, and cryptographic verification of software artifacts. Provenance is highlighted to be verifiable information of the provenance of software (where, when, and how it was created, etc.) in the official SLSA specification [2] [13], whereas The future of artifact verification necessitates a fundamental transition from traditional, cumbersome cryptographic key management to automated, keyless signing architectures. Models like Sigstore will be used to become a standard in the industry [3], based on OpenID Connect (OIDC) identities to create short-lived, temporary certificates. This change in architecture, made possible by lower level building blocks such as Fulcio and Cosign, cryptographically identifies the identity of a developer or a CI/CD runner to a software artifact at the point of compilation. Furthermore, by recording these cryptographic signatures in immutable, append-only transparency logs (such as Rekor), organizations can strictly ensure [11] that the provenance of any container image is verified against tampering before it is orchestrated into a production Kubernetes cluster. A combination of these developments demonstrates that the future software supply chain security will become more evidence-based trust. Consumers will be steadily demanding evidence that it was constructed by an authorized mechanism using known inputs and that it has not been modified prior to release, rather than believing that software is legitimate because it comes out of a well-known source. This is a significant change as it alters software trust to one that is reputation-based instead of one that is verification-based. Provenance and attestations will in the future become common release pipeline elements (at least of container images, binaries, packages, and even SBOM documents themselves). Consequently, the software vendors will have more to show; besides functionality, they will need to establish integrity and traceability.

4.3. Security controls are built within CI/CD pipelines.

The third future trend is that software supply chain protection will further integrate into DevSecOps CI/CD pipelines. NIST SP 800-204D, in particular, is dedicated to software supply chain security integration strategies into CI/CD pipelines [5] and considers build, test, package, and deploy phases of the software supply chain. This is a strong indication that the upcoming protection of software supply chains will be more pipeline-native. Instead of the existing process of manual review or post-hoc checks, organizations will be adopting more and more process controls structured as part of automated processes CI/CD architectures of

the future will be radically changed to a zero-trust, ephemeral build environment based on Policy-as-Code (PaC). With powerful policy engines such as Open Policy Agent (OPA), organizations will enforce security guardrails in a programmable way, preventing unauthorized deployments on a pipeline level. Furthermore, to prevent adversaries from establishing persistence within compromised pipelines, build runners will become entirely immutable and ephemeral—dynamically provisioned for a single execution and immediately destroyed post-build. This transition ensures that even if an attacker successfully pivots from a compromised Git repository, they are operating within an isolated, rapidly decaying infrastructure. This advancement is significant in that software is increasingly being developed and deployed at a high rate, and manual control cannot be scaled to keep up with the current deployment rate. Constant and automatic assurance will therefore become the future of the field, with the future decisions of trust being made during the build and release process rather than many years later when the software has reached the users. Practically, it will lead to CI/CD pipelines becoming one of the primary enforcers of software supply chain security.

4.4. Automated dependency trust score and risk-based adoption.

A fourth direction in the future is the development of automated ways of assessing the reliability of open-source projects and third-party dependencies before adoption. OpenSSF Scorecard has been developed to enable users to determine the level of trust, risk, and security posture [4] of the open-source project, and the design of the tool demonstrates a significant trend in the future of software supply chain security: organizations will demand more attempts to quantify how dependent they are, rather than just the popularity of an open-source project. With the hundreds of direct and indirect dependencies in today's software, human analysis is impossible. So, the focus on dependencies will shift from static evaluations to dynamic trust scoring using generative AI. Security controls will employ large language models and heuristic techniques to monitor open-source repos for deeply hidden code, function obfuscation or backdoors, preventing them from being deployed. The OpenSSF Scorecard will use machine learning to score not just the code, but the maintainer's activity too, so that compromised accounts or governance issues can be identified early in the process. Low-scoring dependencies could be refused for critical systems. This shifts the supply-chain security focus from post-abuse discovery to pre-adoption risk evaluation, minimizing the risks of poorly managed components (with poor hygiene and weak governance), and improving security.

4.5. More focus on safe development maturity and supplier assurance.

A fifth trend in the future is the increasing relevance of secure development maturity and assurance of suppliers. The SSDF delivered by NIST is a well-organized collection of secure software development practices that may be incorporated into software development life cycles, and an initial draft of SSDF version 1.2, published in 2025, indicates that this advice is dynamic as opposed to being static [1]. This implies that the security of the software supply chain in the future will further pressure software producers to exhibit stable, secure

development practices, rather than just assert them. Consequently, the impacted parties, including acquirers, regulators, and enterprise customers, would tend to give greater weight to evidence of safe development, regulated builds, vulnerability management, and software integrity safeguards of suppliers. This is an important development since software supply chain attacks tend to be successful when the development practices are either inconsistent, not well documented, or not enforced. Verifiable maturity in development is likely to influence supplier trust more in the future than vendor reputation. This can affect procurement models, contract requirements, and software assurance reviews in both the government and the private sector.

4.6. Future prospects in the field in general.

All of this points to a future for software supply chain security: the move to proactive and verifiable software security. Previous approaches relied on identifying compromises after they have occurred, while future approaches will priorities vulnerability awareness and signing, automated policy enforcement and ongoing systems and component evaluation. As such, the discipline will not be characterized by any one technology, but by the holistic application of transparency standards, provenance schemes, secure supply chains and build processes, risk scoring of software dependencies, and robust development assurance processes. Such a shift is critical as today's software ecosystems are too complex to rely on trust. Going forward, they will have to validate what they develop, integrate and distribute throughout the software product lifecycle.

Future development	Why is it remerging	Expected security benefit	Supporting framework/source
Wider SBOM adoption and standardization	Organizations need better visibility into direct and transitive software components	Faster vulnerability analysis, stronger procurement review, better incident response	CISA SBOM guidance (CISA) [6]
Provenance, attestations, and signed artifact verification	Trust is shifting from reputation-based assumptions to verifiable evidence-based assumptions	Better integrity assurance, tamper detection, and stronger release trust	SLSA and Sigstore (SLSA) [2] [3]
Security controls embedded into CI/CD pipelines	Modern software is built and released through fast automated pipelines	Continuous enforcement, scalable verification, and earlier risk detection	NIST SP 800-204D (NIST Computer Security Resource Center) [5]
Automated dependency trust scoring	Organizations cannot manually assess every open-source dependency at scale	Proactive risk reduction and better dependency selection	OpenSSF Scorecard (OpenSSF) [4]
Stronger supplier assurance and secure development maturity	Buyers and regulators increasingly expect evidence of secure development practices	Better software assurance, stronger procurement standards, improved accountability	NIST SSDF (NIST Computer Security Resource Center) [1]

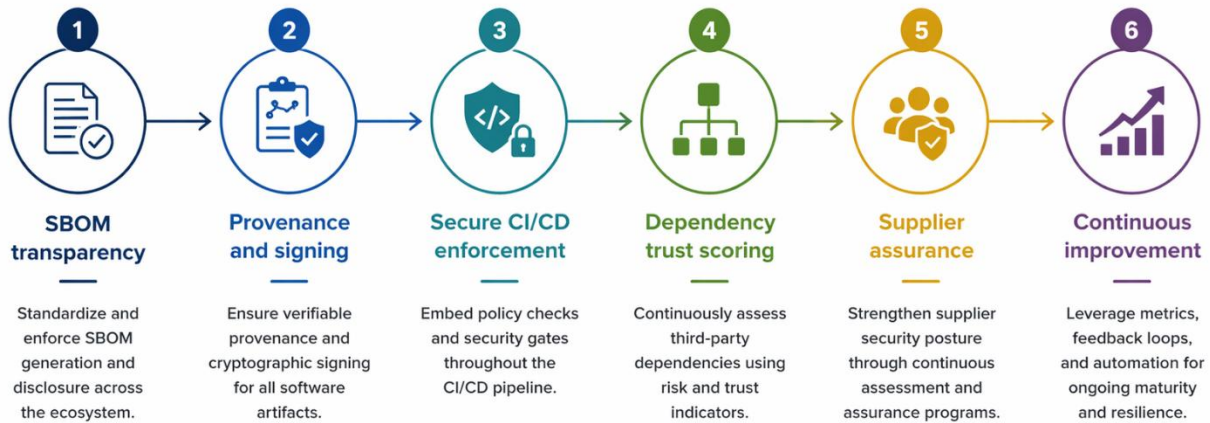


Figure 3: Future direction of software supply chain security.

5. Conclusion

The security of the software supply chain has emerged as an urgent focus in the context of contemporary cybersecurity because contemporary software is orchestrated and delivered across highly interdependent, distributed ecosystems and not along the delimited organizational lines. This report revealed that the topic had shifted to a more inclusive concern where dependencies, repositories, build systems, CI/CD pipelines, software artifacts, and release process integrity were now a broader concern. This more general perspective is reflected in the software supply chain guidance provided by NIST, which considers five stages of the software supply chain: build, test, package, and deploy [5], whereas the NIST SSDF notes that secure software development practices should be incorporated throughout [1] the software development life cycle.

One of the main findings of this report is that trust in software cannot be placed on alone with a reputation of the vendor, popularity of their packages, or untested assumptions about the integrity of the upstream components. The experience of SolarWinds, 3CX, and, more generally, information acquired after the breach [7] [8] of the supply chain have revealed that attackers are resorting to relied development and delivery channels, in which they can find the numerous downstream consumers simultaneously. As a result, software supply chain security has shifted from implicit trust to its need for transparency, integrity, provenance and assurance. Positive outcomes include formalized security practices, such as secure software development processes; SBOM visibility; and improved DevSecOps practices.

The report also highlights emerging trends: increased SBOM usage; enhanced provenance and artifact verification; greater integration up the pipeline; automated dependency trust assessment; and increasing demands for supplier secure development maturity. In summary, software supply chain security is moving towards a paradigm where software not only needs to work but also demonstrate its provenance, development and integrity. In short, software supply chain security is now essential, rather than specialist, for the development of reliable, transparent and trusted systems.

References

- [1] M. Souppaya, K. Scarfone and D. Dodson, Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities, National Institute of Standards and Technology, NIST SP 800-218, Feb. 2022.
- [2] SLSA, SLSA Specification v1.2.
- [3] Sigstore, Overview.
- [4] O. S. S. Foundation, OpenSSF Scorecard.
- [5] Chandramouli, Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines, National Institute of Standards and Technology, NIST SP 800-204D, Feb. 2024.
- [6] C. a. I. S. Agency, 2025 Minimum Elements for a Software Bill of Materials (SBOM), Aug. 2025.
- [7] C. a. I. S. Agency, Supply Chain Compromise, Jan. 7, 2021.
- [8] C. a. I. S. Agency, Supply Chain Attack Against 3CXDesktopApp, Mar. 30, 2023.
- [9] C. a. I. S. Agency, Lessons from XZ Utils: Achieving a More Sustainable Open Source Ecosystem, Apr. 12, 2024.
- [10] E. U. A. f. Cybersecurity, ENISA Threat Landscape for Supply Chain Attacks, Jul. 2021.
- [11] Sigstore, Rekor.
- [12] in-toto, What is in-toto?, Nov. 22, 2024.
- [13] SLSA, Provenance.